Applied Software Development Lab

Lab Handout 11: Levels, lives, variety, and control

Learning objective: Bringing a software project to a playable, impressive conclusion under realistic conditions (read: shortly before the deadline)

Note: This time, the steps contain no explicit prompts to commit your current state to the repository. You can now handle that on your own – and rightly so!
In case you do hesitate for a moment: Step 13 on the last page has a little reminder and instructions.

Preparation

As usual, we work with two terminals:
- **left**: edit the source code
- **right**: test the game

Cloning the repository:

```
git clone https://gitlab.uni-ulm.de/USERNAME/asp-ulm
cd asp-ulm/python
python breakout.py
```

As always, **take a moment to familiarize yourselves** with the features your game currently supports.

If **unexpected crashes occur** – and you are **lucky enough** to discover them now – it is best to fix them right away. **We are happy to help.**

Step 1: Bombs are not just for decoration

When the paddle is hit by a bomb, it should be **out of action** for a certain time. In this step, we first only check whether a bomb hits the paddle. If so, the bomb should detonate.

To do this, you can **proceed similarly** to the check whether a ball hits the paddle.
If you want to do it really well: take a look at `ball.py`. There, the `move` method checks whether the **center** of the ball crosses the **top edge** of the paddle.

If you proceed analogously, you can adapt the `move` method in `bomb.py` accordingly. The code is even **a bit simpler** here, because our bombs always fall **straight down**.

When the paddle is hit, you simply set the attribute `self.hit = True`. The `draw` method takes care of the rest – **without any changes**.

Your `move` method in `bomb.py` could then look like this, for example:

```
    def move(self):
        if self.y < 0 or self.y > Game.height:
            self.alive = False
            return
        new_y = self.y + self.vy
        if self.y < Paddle.y-10 and new_y >= Paddle.y-10:
            if self.x >= Paddle.x and self.x <= Paddle.x + Paddle.w:
                self.hit = True
        self.y = new_y
```

Task: Test whether, after your change, the bombs explode when they hit the paddle.

Step 2: Bombs are bad for the paddle

When the paddle has been hit, this should have **consequences**: for a certain time, it is **out of action** – and, in particular, **not visible**. Here is a small **template** showing how the feature can be implemented **at least partially**. You will extend the rest yourselves in the course of the task.

🧱 In `paddle.py`

The paddle gets a new static attribute `damage`, which is initialized with `0`. It indicates **for how many frames** the paddle cannot be used. The class `Paddle` thus begins like this, for example:

```
class Paddle:
    c = (0, 0, 255)
    w, h = 100, 30
    x, y = None, None
    damage = 0
```

💥 New method `hit`

The method `hit()` "destroys" the paddle – **but only if it is still intact**. If it is already damaged, nothing happens.

```
    def hit():
        if Paddle.damage == 0:
            Paddle.damage = 90
            return True
        return False
```

Note: The method returns `True` if the paddle has just been damaged. Otherwise, it returns `False` (e.g., if it is already broken anyway).

✏️ Adapting the `draw` method

If the paddle is damaged, it is not drawn – but the damage is reduced:

```
    def draw():
        if Paddle.damage > 0:
            Paddle.damage -= 1
            return
        pygame.draw.rect(Game.screen, Paddle.c,
                         (Paddle.x, Paddle.y, Paddle.w, Paddle.h))
```

Task

1. Adapt the code in `paddle.py` as described above. As always, try to understand exactly what happens. Now, of course, `Paddle.hit()` should be called when an intact paddle is hit by a bomb.
   To do this, change the line `self.hit = True` in `bomb.py` to `self.hit = Paddle.hit()`. This way, `self.hit` is only set to `True` if the paddle was actually damaged by the bomb hit.

2. The paddle is now invisible for a while after it has been hit – but it is still there. That is, even in its damaged state, it can still reflect balls.
   Hence, two subtasks:
   (a) (hard) Observe the behavior
   First, try to **consciously observe** the problem before you fix it. **Can you make a ball bounce off an invisible paddle?** That is not easy at all – but this is exactly the kind of behavior a good test team should discover
   (b) (easy) Fix the problem
   In `ball.py`, adapt the place where it is checked whether the ball collides with the paddle. Add an additional condition there so that the ball **is only reflected** if the paddle **is intact**.

3. Now also fix the problem that you can currently still fire rockets with a destroyed paddle.

© Michael Lehn | CC BY-SA 4.0

Step 3: Long live variety (part 1 of 3)

Currently, our bricks are still rather **monotonous**: they are either **there or gone** – nothing more. That is about to change. There should be **different kinds of bricks**, for example:

- Bricks that you have to **hit twice** (only then do they disappear)
- Bricks that bring **a new ball** into the game when destroyed
- Bricks that give the paddle **a temporary protective shield** – after that, you are immune to bombs for a while
- And maybe more…

Idea: numbers for brick types

The type of a brick can initially be encoded with numbers:

- `0` → no brick
- `1` → normal brick (what we have had so far)
- `2` → brick that has to be hit twice
- `3` → brick that creates a new ball
- `4` → brick that activates a protective shield

This has the advantage that, as before, you can work with a `numpy` array or with comparison operators.

But: **numbers alone** are **not particularly expressive** – you quickly forget what 3 or 4 stood for.

Remedy: `enum` types

Here comes a mini crash course – entirely in the style of: **"Learning by using and playing around."**

Create a new file named `bricktype.py` with the following content:

```
 1 from enum import IntEnum
 2
 3 class BrickType(IntEnum):
 4     NONE = 0
 5     NORMAL = 1
 6     BALL = 2
 7     SHIELD = 3
 8     TWICE = 4
 9
10     def label(self):
11         return f"Dummy label for {self}"
12
13 def main():
14     print(max(BrickType))
15     print(BrickType.BALL)
16     print(BrickType.BALL.label())
17     foo = BrickType(4)
18     print(foo)
19     print(foo.label())
20
21
22 if __name__ == "__main__":
23     main()
```

© Michael Lehn | CC BY-SA 4.0

Explanations

- In **line 3**, `BrickType` is declared as an extension of `IntEnum`. At this point, we will **not go into detail** about what this means – **except** in a later **subtask**, in which you remove the `(IntEnum)` once to see **what suddenly no longer works**.
- In **lines 4–8**, the different **kinds of bricks** are defined as constants with **meaningful names** – much more readable than plain numbers.
- In **lines 10–11**, a method label is defined that, for now, only returns a **placeholder text** – but already shows **what is possible with enums**.

Task

1. Try it out with `python bricktype.py`. Observe the following:
   - `max(BrickType)` returns the largest enum member – in this case, `BrickType.TWICE`
   - You can use `BrickType.BALL` instead of the number `2` – much more expressive!
   - `BrickType.BALL` is not simply a number: for example, you can apply `.label()` to it
   - With `BrickType(4)`, you can convert an ordinary integer value back into a `BrickType` object

2. **What do we gain from this `(IntEnum)`?**
   In **line 3**, remove the `(IntEnum)` and run the program again:
   - **Error in line 14**. The function `max()` can no longer be applied to the class.

   Comment out line 14 and run the program again:
   - Now you get a new **error in line 16**. `BrickType.BALL` is now just a number, and the method `label` can no longer be applied.

   What we learn about the effect of `(IntEnum)`:
   - Attributes such as `BrickType.BALL` are not ordinary `int` values but objects of type `BrickType`
   - These objects have methods such as `.label()`, which you define yourself
   - In addition, you "inherit" certain things, e.g., that you can write `max(BrickType)`

   Undo all changes: you are welcome to look into the exact details on a **quieter day** – today is about **understanding by trying things out**, not about the last Python detail 😉

3. **Extending the `main()` function and improving `label()`**
   Change the function main() as follows:

   ```
   def main():
       for i in range(0, max(BrickType)):
           bt = BrickType(i)
           print(f"i = {i}, bt = {bt}, bt.label = {bt.label()}")
   ```

   Run the program and follow what happens. Now change the method `label()` so that – depending on the value of `self` – it returns the following text:
   - `"Ball"` for `BrickType.BALL`
   - `"Shield"` for `BrickType.SHIELD`
   - `"Twice"` for `BrickType.TWICE`
   - In all other cases, the empty string `""`

   👉 This is a small exercise in `if`–`elif`–`else`.

© Michael Lehn | CC BY-SA 4.0

Step 4: Long live variety (part 2 of 3)

Now the class BrickType is **brought into the game very gently**. At first, it is **only about the display**: bricks should **differ visually** by having a small **label** text printed on them.

Important: these bricks **do not behave differently yet** – that comes later.

🧱 In `wall.py`

First, import our new class with `from bricktype import BrickType`.

🛠️ Adapting the `init` method

Then change the method `init` as follows:

```
12      def init():
13          for i in range(4, 8):
14              for j in range(0, Game.brick_cols):
15                  Wall.brick[i, j] = random.randint(1, max(BrickType))
```

Previously, all cells there were bluntly filled with `1` – now each cell gets a random `BrickType` value (from `1` up to and including `4`).

🖼️ Extending the `draw` method

Now we make the new brick types visible. In the `draw` method, add the following code starting at **line 45**:

```
33      def draw():
34          for i in range(0, Wall.brick.shape[0]):
35              for j in range(0, Wall.brick.shape[1]):
36                  if not Wall.brick[i, j]:
37                      continue
38                  w = Game.brick_width
39                  h = Game.brick_height
40                  x = j * w
41                  y = i * h
42                  r = (x + 1, y + 1, w - 1, h - 1)
43                  pygame.draw.rect(Game.screen, (223, 105, 165), r)
44
45                  label = BrickType.label(Wall.brick[i, j])
46                  font = pygame.font.SysFont(None, 24)
47                  text = font.render(label, True, (255, 255, 255))
48                  text_rect = text.get_rect(center=(x+w//2, y+h //2))
49                  Game.screen.blit(text, text_rect)
```

Here, the label is now drawn onto the rectangle:

- **Line 45**: calls `BrickType.label(...)` and returns an ordinary string.
- **Line 46**: creates a `Font` object (font size 24, default font).
- **Line 47**: converts the string into a **bitmap image** (font color: white, with antialiasing).
- **Lines 48–49**: draws the text **centered** in the brick's rectangle.

© Michael Lehn | CC BY-SA 4.0

Task

1. Test whether your adaptation works – i.e., whether some bricks now have labels. You should be able to read, e.g., `"Ball"`, `"Shield"`, or `"Twice"` on individual bricks.
2. Currently, **too many special bricks** appear – this looks a bit cluttered. It would be better if, with a certain probability (e.g., **20 %**), a brick got a special function and otherwise simply became a **normal** brick (`NORMAL`).

   Suggestion:

   ```
   if random.randint(1, 100) <= 20:
       ty = random.randint(1, max(BrickType))
   else:
       ty = 1
   Wall.brick[i, j] = ty
   ```

   Note: In the special selection, `BALL`, `SHIELD`, and `TWICE` are then equally likely – which is perfectly fine for our purposes.
3. It is not dramatic – but still a small break in style – that a new font object is created in every single `draw()` iteration. Even if the performance loss is barely measurable: good code has manners.

   It would be better if the font object were created once, centrally, in the Game class.

   🔧 **Here's how**

   In `game.py`, add a new attribute to the class Game:

   ```
   class Game:
       # ... as before ...
       font = None
   ```

   Then adapt the `init()` method as follows:

   ```
       def init():
           # ...
           pygame.init()
           # ...
           Game.font = pygame.font.SysFont(None, 24)
   ```

   Important: The font object must not be created until after `pygame.init()` has been called!

   Afterwards, in `wall.py`, you can replace the line

   ```
   font = pygame.font.SysFont(None, 24)
   ```

   with:

   ```
   font = Game.font
   ```

4. **Optional polish: color by type**
   In `bricktype.py`: add a method color to the class `BrickType` that – analogous to the method `label()` – returns a suitable color for each brick type.

   In `wall.py`: adapt the `draw()` method so that the color for the brick is now obtained from the `color()` method before drawing:

   ```
   c = BrickType.color(Wall.brick[i, j])
   pygame.draw.rect(Game.screen, c, r)
   ```

© Michael Lehn | CC BY-SA 4.0

Step 5: Long live variety (part 3 of 3)

Our special bricks should now not **only look different** but also **have an effect** when you hit them.

We approach this **step by step** – and start right away with the "most complicated" case: the **bricks that have to be hit twice**.

🧠 Idea: changing the kind of brick on a hit

Bricks of type `TWICE` should **turn into** `NORMAL` **on the first hit**,
and then **disappear** on the second hit.

🧱 In `bricktype.py`

Extend the class `BrickType` with a method `effect()` that returns what a brick should turn into when it is hit:

```
def effect(self):
    if self == BrickType.TWICE:
        return BrickType.NORMAL
    else:
        return BrickType.NONE
```

🔧 In `wall.py`

In the method collision(), a brick that was hit used to be simply deleted, e.g., like this:

```
if Wall.brick[i2, j2]:
    Wall.brick[i2, j2] = 0
```

Now replace this with:

```
if Wall.brick[i2, j2]:
    Wall.brick[i2, j2] = BrickType.effect(Wall.brick[i2, j2])
```

This way, each brick decides for itself what happens to it when it is hit.

Task

1. Test whether bricks of type `TWICE` now only disappear on the second hit.

   Pro tip: To test this specifically, you can **temporarily set all bricks to** `TWICE` in the `init()` method of `Wall`, e.g.:

   ```
   Wall.brick[i, j] = 4 # ty
   ```

   This saves you the tedious search for the right brick – and you can see immediately whether the behavior is correct.
2. **The easy case? Think again.**
   Actually, everything would now be quite easy: when a brick of type `BALL` is hit, **a new ball should come into play**.

   Sounds simple: just create a new `Ball` object, append it to `Game.balls`, done! **But think again**. Python unfortunately makes the **actually easy case** complicated.

   #### 🧪 Experiment: what goes wrong?

   In `bricktype.py`, add the following line at the top:

   ```
   from ball import Ball
   ```

   Make no other changes – and start the game. It will not work.

© Michael Lehn | CC BY-SA 4.0

Step 6: Cyclic dependencies

What **went wrong** at the end of the last step? You have created the following **import chain**:
- `bricktype.py` imports `ball.py`
- `ball.py` imports `wall.py`
- `wall.py` imports `bricktype.py`

🐈‍⬛ **And so the cat is chasing its own tail.**

In an "ideal" object-oriented language, every class can **refer to other classes arbitrarily** – after all, that is exactly the point of modular development. But in **Python**, **all of the code is executed immediately** on import – and because Python does not cleanly separate **interface** and **implementation**, **cyclic dependencies** lead to technical problems.

🩹 **Makeshift solutions (not recommended)**

An unclean solution would be to replace (almost) **all** `from ... import ...` statements with plain `import ...` statements and to use, e.g., `module.Class` later on.
This **circumvents** the problem – but it is not pretty and breaks with the style.

Another (even more questionable) option would be to write the `from ball import Ball` directly into the method where the ball is needed – e.g., into `BrickType.effect`.
That would be a quick solution, but honestly: it breaks my heart. On every call, Python would check whether the module needs to be loaded. Even if this is technically cached: there is no need to trample all over performance.

🧼 **A somewhat cleaner solution: controlling the imports**

We break the import cycle by no longer importing `BrickType` directly in `wall.py`, but passing it to `Wall` at runtime instead.

🧱 **Changes in `wall.py`**

At the top, remove the line: `from bricktype import BrickType`

Replace the beginning of the class with:

```
class Wall:
    brick = numpy.zeros((Game.brick_rows, Game.brick_cols))
    BrickType = None
```

Adapt the method `init()` so that `BrickType` is passed and stored:

```
def init(BrickType):
    Wall.BrickType = BrickType
    for i in range(4, 8):
        for j in range(0, Game.brick_cols):
            if random.randint(1, 100) <= 20:
                ty = random.randint(2, max(Wall.BrickType))
            else:
                ty = 1
            Wall.brick[i, j] = ty
```

Change the remaining code in `wall.py` so that `Wall.BrickType` is always used instead of `BrickType`.

🔧 **Changes in `breakout.py`**

As usual, import `from bricktype import BrickType` at the top.
Instead of `Wall.init()`, now call `Wall.init(BrickType)`.

Make the changes as described and test whether everything still works. If so, you now have a clean, cycle-free design and are ready for the next step

© Michael Lehn | CC BY-SA 4.0

Step 7: Back to features

Now that we have elegantly solved the problem of cyclic dependencies, we can finally import everything we need in `bricktype.py` –

(at least everything except from `breakout.py` 😉).

For the next step, this is enough:

```
from ball import Ball
from game import Game
```

🔧 Adapting the `effect()` method

Now you can extend the method `effect()` in the class `BrickType` as planned:

```
def effect(self):
    if self == BrickType.TWICE:
        return BrickType.NORMAL
    elif self == BrickType.BALL:
        Game.balls.append(Ball(100, 0, -3, 4))
        return BrickType.NONE
    else:
       return BrickType.NONE
```

🧪 Test

With this change, destroying a brick of type `BALL` should bring a **new ball** into the game.

To test this specifically, you can – as in earlier tasks – temporarily initialize the bricks in `wall.py` like this:

```
Wall.brick[i, j] = 2 # ty
```

Then you will see immediately whether a new ball is created on a hit.

Task

While testing, you may be pleased only briefly that a new ball comes into play. Unfortunately, it is created **above the brick wall** – and then makes for a rather boring game, because it simply clears **bricks from top to bottom** until a big hole has been punched through.

🧠 Idea

We change the game behavior so that a newly created ball is initially inactive – i.e., it flies through the bricks until it is hit by the paddle. Only then does it become active and can destroy bricks.

🔧 Implementation in `ball.py`

1. In the constructor (`__init__`), the ball gets a new attribute:
   `self.active = False`
2. In the `move()` method:
   - Only call `Wall.collision(...)` if `self.active == True`
   - When the ball hits the paddle, set `self.active = True`

Start the game and check whether a new ball now first falls through the brick wall and can destroy bricks only after contact with the paddle.

© Michael Lehn | CC BY-SA 4.0

Step 8: Protective shield

Now the paddle gets yet another feature: a protective shield that prevents bomb damage for a certain time.

🧱 Preparation in `paddle.py`

First, the class Paddle is extended with a new attribute `shield`, similar to `damage`.
It counts down as long as the shield is active:

```
class Paddle:
    # ...
    damage = 0
    shield = 0
```

💥 Adapting the method `hit()`

The shield protects against damage – so `hit()` has to be adapted like this:

```
def hit():
    if Paddle.damage == 0 and Paddle.shield == 0:
        Paddle.damage = 90
        return True
    return False
```

✨ New method `set_shield()`

This method activates the shield:

```
def set_shield():
    Paddle.shield = 300
```

🎨 Cosmetics in `draw()`

When the shield is active, the paddle is drawn in white, for example:

```
def draw():
    if Paddle.damage > 0:
        Paddle.damage -= 1
        return
    if Paddle.shield > 0:
        Paddle.shield -= 1
        c = (255, 255, 255)
    else:
        c  = Paddle.c
    pygame.draw.rect(Game.screen, c,
                     (Paddle.x, Paddle.y, Paddle.w, Paddle.h))
```

Task

1. Make the changes described above in `paddle.py`.
2. Then adapt the method `effect()` in `bricktype.py` so that bricks of type `SHIELD` are supported.

   You know the procedure by now:
   - **Think about** what you additionally need to import.
   - **Think about** how you can quickly test whether the feature works.

If everything works, the paddle should be invulnerable to bombs for 10 seconds after destroying a `SHIELD` brick – and glow bright white while doing so ⚡

© Michael Lehn | CC BY-SA 4.0

Step 9: Countermeasures

Wouldn't it be great if you could also **actively defend yourself against bombs** – that is, simply **shoot them down** before they hit you?

🧠 Oh dear.

Yes, that would be great… But **should you really, at the last minute,** build in yet another new feature instead of working through important items on the to-do list?
No. **You shouldn't.**

🤷 Just like in real life…

… we do it **anyway**. And actually, it is not that big a deal – it should **work exactly the same way** as shooting a space invader out of the sky.

🛠️ The existing code in `breakout.py`

This is how space invaders are currently hit by balls or rockets:

```
for s in Game.ships:
    for item  in Game.balls + Game.rockets:
        dist = math.hypot(item.x - s.x, item.y - s.y)
        if dist <= 15:
           s.hit = True
           item.hit = True
```

Thanks to the way of thinking sharpened in the HM (higher mathematics) lectures, we immediately recognize: if bombs are to be destroyed **exactly like** ships, then we simply have to **iterate over the bombs** as well.

So it is enough to replace the loop:

```
for s in Game.ships:
```

with:

```
for s in Game.ships + Game.bombs:
```

Task

Obvious, right? **Just do it**. 😄

Step 10: Because there is still a little room here

When all space invaders have been shot down, it gets boring – after all, we want to play Breakout **and** Space Invaders. So let us quickly add the feature that a new space invader appears for every brick that is hit.

In `bricktype.py`, add the following line at the very beginning of the method `effect()`:

```
Game.ships.append(Ship(0))
```

That should actually be enough.
Just test it.
If some import thingy is missing, an error message will tell you 😉

© Michael Lehn | CC BY-SA 4.0

Step 11: Take a look at the clock …

… maybe we should start with one of the important features. So let us pause briefly and think: **What absolutely belongs in a game like Breakout + Space Invaders?** Perhaps that you only have a limited number of lives – and fight to get to the next level before all lives are used up. In other words, exactly what makes university studies, with their limited number of permitted exam attempts, so exciting.

Okay, how do we start? In any case, we need to **keep track** of two important things:
- the number of **lives**
- the current **level**

🧱 In game.py

Add two new attributes to the class Game:

```
class Game:
    # ... other attributes as before
    lifes = 0
    level = 0
```

🖼️ And then: showing where things stand

So that you can also see what situation you are in, we display `level` and `lifes` in the top left corner of the window. Luckily, you have already seen today how to render text – so you can, e.g., add the following at the end of the `draw()` method:

```
    def draw():
        # ... everything as before
        label = f"Level {Game.level} Lifes {Game.lifes}"
        text = Game.font.render(label, True, (255, 255, 255))
        text_rect = text.get_rect(topleft=(10, 10))
        Game.screen.blit(text, text_rect)
```

🧪 Test

Once you have made these two changes, you should see in the top left corner that you have **0 lives** and are at **level 0**. Sounds sobering at first – but: **it is a start**.

🔁 And now: an endless loop

Now we want to let the game **keep running robustly**, i.e., build in a simple restart mechanism:

- When you have no lives left, the game starts over from the beginning. For example, with 3 lives at level 1
- When there is no ball left in the game, you lose a life (if you still have one), and a new ball comes into play.

🔧 Implementation in `breakout.py`

Start with an empty ball list, `Game.balls = []`, and implement the idea inside the main loop (e.g., directly after `while running:`) as follows:

```
    if len(Game.balls) == 0:
        if Game.lifes > 1:
            Game.lifes -= 1
        else:
            Game.lifes = 3
            Game.level = 1
            Wall.init(BrickType)
        Game.balls.append(Ball(400, 0, -2, 6))
```

© Michael Lehn | CC BY-SA 4.0

Task

1. Implement the changes described above and test the game. You should now start with 3 lives at level 1 (that is already an improvement). When all balls have been lost, you lose a life.

2. Great, you can now lose lives – but not gain any yet. That may be how it is in real life, but in a game you want more. For example: that you move up to the next level when all bricks have been cleared – and then face an even greater challenge, such as an additional row of bricks.

   To do this, change the `init` method in `wall.py` so that the number of brick rows corresponds to the current level:

   ```
   def init(BrickType):
       Wall.BrickType = BrickType
       Wall.brick = numpy.zeros((Game.brick_rows, Game.brick_cols))
       for i in range(4, 4 + Game.level):
           for j in range(0, Game.brick_cols):
               if random.randint(1, 100) <= 20:
                   ty = random.randint(2, max(Wall.BrickType))
               else:
                   ty = 1
               Wall.brick[i, j] = ty
   ```

   In `breakout.py`, first remove the old initialization with `Wall.init(BrickType)`. Instead, you can now regularly check in the main loop how many bricks are still in play. When none are left, you move up a level:

   ```
   if Wall.brick.sum() == 0:
       Game.level += 1
       Wall.init(BrickType)
   ```

   The method `sum` comes from `numpy`, which – as you know – is a wrapper for a very efficient C/C++ library for numerical linear algebra. Computing the sum of all matrix elements is extremely efficient. So it is not worth counting separately on the Python side how many bricks are still left. The code is short, clear, and performant.

   Make these changes and test the game. You now start with one row of bricks. If you manage to clear it completely, you move up a level and face a new wall with an additional row. If you have lost all lives, you start again with 3 lives at level 1.

Step 12: Since there is still room, here's something for your ears

Unfortunately, the PCs in the lab rooms have no speakers. But there are headphones! And anyone who feels like giving the game a bit more punch can now add sound effects.

With pygame, this is very easy. A sound object is created from an audio file – and you can play it with .play() whenever you feel like it.

For example, in game.py in the init method:

```
Game.exploding = pygame.mixer.Sound("data/exploding.wav")
Game.bounce = pygame.mixer.Sound("data/bounce.mp3")
Game.launch = pygame.mixer.Sound("data/launch.wav")
```

When, e.g., a rocket is fired, the matching clip can then be played like this:

```
Game.launch.play()
```

There is no more room for explanations here – but that does not matter, because at this point, your enthusiasm for experimenting is called for. Think about where else sound effects could be played.

© Michael Lehn | CC BY-SA 4.0

Step 13: Oh yes, the repository …

If anyone has made it this far: it is about time to commit the changes to the repository – unless you have already come up with that idea yourselves anyway 😉

```
git add *.py
git commit -a -m "about time"
git push
```

Step 14: And then there is the matter of the controller

In the lab, we have provided you (unless we forgot) with a game controller with a BNO055 sensor. With it, you can steer the paddle casually with a flick of your wrist.

It should be surprising – or maybe not – that you did not have to adapt your Python code for reading the serial interface at all. Why? Because it makes no difference at all whether the microcontroller reads a potentiometer or another sensor – as long as it sends the data over the serial interface using the same protocol, everything works as before.
If you want the matching microcontroller code for the BNO055:

```
git clone git@github.com:michael-lehn/asp-ulm-bno055.git
```

The Makefile works the way you know it by now:

- `make` compiles the code
- `make upload` compiles and flashes
- `make monitor` starts the serial monitor

Step 15: And here there would still be room …

… for a long to-do list with refactoring, clean code, a menu, a high score, and options.
Or for a few closing words about the path we have taken.

But honestly: I like neither to-do lists nor goodbyes 😄

© Michael Lehn | CC BY-SA 4.0